

codeHive

Hash Sets

UNIQUENESS & EFFICIENT LOOKUP

Data Structures: Module 22

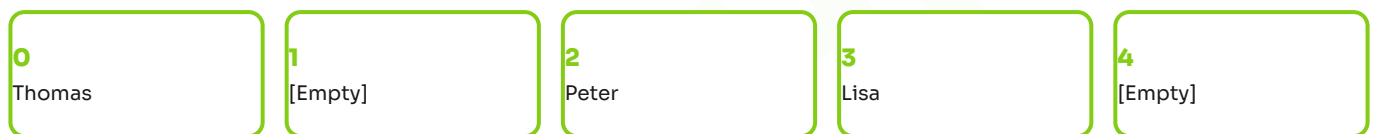
codeHive

1. The Logic of Unique Sets

A **Hash Set** is a specialized Hash Table that stores unique elements. Its primary purpose is high-speed membership testing: "is this item in the collection?"

Core Definitions:

- **Hash Code:** A numeric signature generated from a key's value.
- **Bucket:** A storage container in the array. Each index is a bucket.
- **Uniqueness:** A Hash Set automatically rejects duplicate entries.



2. Direct Access Performance

Hash Sets typically operate at **$O(1)$** time complexity. However, this depends on how well the Hash Function distributes data.

Complexity Analysis:

- **Average Case: $O(1)$** - Data is spread evenly across buckets.
- **Worst Case: $O(n)$** - All items collide in a single bucket (forming a linked list), turning the search into a linear scan.

Pro Tip To maintain speed, ensure the number of buckets is roughly equal to the number of expected elements.

3. Python Implementation

The following SimpleHashSet class demonstrates collision handling via chaining (using lists within buckets).

```
class SimpleHashSet:
    def __init__(self, size=100):
        self.size = size
        self.buckets = [[] for _ in range(size)]

    def hash_function(self, value):
        # Convert string to index
        return sum(ord(char) for char in value) % self.size

    def add(self, value):
        index = self.hash_function(value)
        if value not in self.buckets[index]:
            self.buckets[index].append(value)

    def contains(self, value):
        index = self.hash_function(value)
        return value in self.buckets[index]
```

4. Summary of Operations

Operation	Description
add(val)	Generates hash, checks for duplicates, and inserts.
contains(val)	Goes directly to the bucket via hash to check existence.
remove(val)	Locates the bucket and removes the specific item.

© 2026 codeHive Academy • Hashing & Set Theory